LENGUAJE PARA LA SIMULACION DIGITAL DE SISTEMAS DINAMICOS CONTINUOS

Ing. Víctor Trejo
Ing. Pablo Padrón

Cos. Manuel Fernández
María M. Villoch
Oscar Fabelo
Lázaro Hernández

El presente trabajo describe un lenguaje para la simulación digital de sistemas dinámicos contínuos por medio de la Computadora Digital CID-201-A, usando los métodos de programación de las Computadoras Analógicas, elaborado por el Grupo de Programación y Control de la Escuela de Ingeniería Eléctrica de la Universidad de La Habana.

El trabajo señala las características del Lenguaje, los bloques que a disposición del usuario pone éste, el formato de las instrucciones y el método para la elaboración de programas con el mismo. También se expone la secuencia de operaciones que realiza el Lenguaje para llevar a cabo una simulación.

Se desarrolla un ejemplo de la programación de una simulación particular, mostrándose el programa resultante y los resultados obtenidos.

Se incluye una relación de los errores de programación que detecta el Lenguaje, y en las conclusiones se comparan las posibilidades del Lenguaje con relación a la computadora analógica.

INTRODUCCION

Los lenguajes de simulación constituyen una poderosa herramienta para la ingeniería moderna, al permitir el uso de las computadoras digitales electrónicas a la simulación de sistemas, sin necesidad de disponer de conocimientos profundos de métodos numéricos y programación.

El presente trabajo describe un lenguaje para la simulación digital de sistemas dinámicos contínuos por medio de la computadora digital *CID-201-A*, usando los métodos de programación de las computadoras analógicas. Este lenguaje permite combinar la gran precisión de la computadora digital con la flexibilidad de la analógica, lográndose además, la estrecha relación existente en ésta, entre los elementos físicos que componen un sistema y la implementación de la simulación.

Este lenguaje ha sido elaborado por el Grupo de Programación y Control de la Escuela de Ingeniería Eléctrica de la Universidad de la Habana, y servirá como una ayuda en el trabajo futuro de dicho Grupo y como base, para la introducción del estudio de las técnicas de simulación digital en distintas especialidades dentro de Ingeniería Eléctrica.

CARACTERISTICAS DEL LENGUAJE

El lenguaje pone a disposición del usuario, de forma similar a la computadora analógica, un conjunto de bloques cuya relación salida-entrada se corresponde con la de elementos que componen los sistemas físicos reales. La interconexión de dichos bloques, siguiendo la metodología que establece el lenguaje, permite la modelación de sistemas relativamente complejos para su estudio por medio de la computadora digital *CID-201-A*.

La programación utilizando el lenguaje es simple, especialmente para aquellas personas que conocen la programación en computadora analógica, aunque no se requiere este conocimiento para el uso del lenguaje.

Los resultados de las simulaciones pueden obtenerse en forma de listado de hasta tres variables del sistema de simulación contra el tiempo, o en forma de gráfico de una variable contra el tiempo, en este caso se obtiene también el listado de la variable graficada. El formato de impresión es de cinco enteros con cinco cifras decimales, por lo que sólo se requiere escalar en magnitud, cuando los valores de los resultados son menores de 10^{-5}, los resultados con seis o más cifras enteras se imprimen, pero sin alinearlos por el punto decimal. Los resultados se obtienen con cinco cifras significativas.

En ningún caso se requiere escalar en tiempo.

El intervalo de impresión de los resultados, así como el tiempo final de corrimiento del problema y el intervalo de integración, son seleccionables por el programador y pueden ser modificados sin hacer un nuevo programa por medio del teclado decimal de la máquina, al final de correr el programa o deteniendo el mismo en el instante que se desee.

De la misma forma pueden modificarse los parámetros de los componentes del sistema, al igual que en la computadora analógica.

El lenguaje emplea el método de Euler modificado para la solución de las ecuaciones diferenciales presentes en la simulación.

Para efectuar una simulación se perfora en cinta de papel el programa elaborado y se coloca la cinta en el lector, iniciándose el proceso por medio de una tecla de la máquina. En caso de errores en el programa elaborado, la máquina pasa a ALTO DINAMICO y se imprime en el teletipo la palabra "Error" seguida de un número que identifica el tipo de error cometido. El Anexo 1 relaciona los distintos errores detectables por el lenguaje.

El programa que compone el lenguaje se carga por medio de la grabadora de CASSETTE y pueden hacerse las simulaciones que se requieran sin necesidad de recargarlo.

ANEXO 1

ERRORES DETECTABLES POR EL LENGUAJE

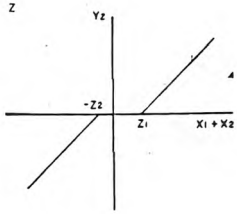
Número de referencia repetido, mayor que 127 o menor que 1.
Doble punto decimal.
Doble signo.
Símbolo del teletipo no permitido.
Código alfabético no permitido.
Instrucción con mayor o menor cantidad de parámetros que los requeridos.
Más bloques de un tipo que los permitidos.
Bloque con más salidas que las permitidas.
Bloque no conectado.
Intervalo de impresión menor que el intervalo de integración.
Tiempo final menor que el intervalo de impresión.
Tiempo de retraso de un elemento de retraso puro menor que un semi-intervalo de integración.
Limitador o zona muerta con parámetro negativo.
Generador de funciones mal escalado.
Modificación por teclado o bloque no existente en el programa original.
Modificación por teclado a bloque sin parámetro.

BLOQUES QUE COMPONEN EL *LENGUAJE*

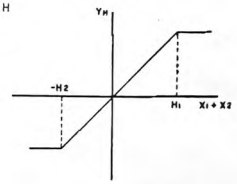
El lenguaje se compone de los bloques siguientes:

Nombre	*Código*	*Relación salida-entrada*
1) Función forzante...	F	$Y_F = A + Bt + Ct^2 + D \operatorname{sen}(2\pi Et + F)$
2) Sistema de primer orden	P	$\dot{Y}_P + aY_P = X_1 + X_2$
3) Sistema de segundo orden.....	W	$\ddot{Y}_W + bY_W + cY_W = X_1 + X_2$
4) Retraso puro.....	T	$Y_T = U\,(t - T)\,[X_1 + X_2]\,(t - T)$
5) Sumador	C	$Y_C = X_1 + X_2$
6) Multiplicador	X	$Y_X = X_1 X_2$
7) Divisor	O	$Y_O = X_1 \div X_2$
8) Inversor	V	$Y_V = -(X_1 + X_2)$
9) Nodo	Y	$Y_Y = X_1 + X_2$
10) Amplificador	K	$Y_K = K\,(X_1 + X_2)$
11) Derivador	Q	$Q_Q = \dot{X}_1 + \dot{X}_2$
12) Relé	B	$Y_B = \text{signo}\,(X_1 + X_2)$

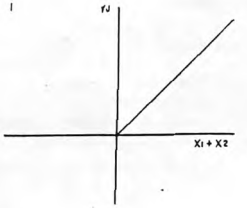
13) Zona muerta.. Z



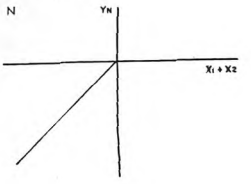
14) Limitador .... H



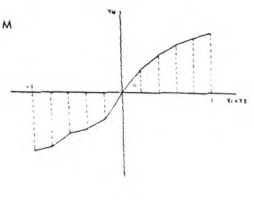
15) Diodo (+).... I



16) Diodo (—).... N



17) Generador de funciones..... M



Todos los bloques a excepción del 1 pueden usar una o dos entradas. Las salidas de los bloques del 1 al 4 no tienen límite, mientras que el bloque 8 emplea dos salidas y el resto solo una.

El número total de bloques que pueden ser usados en una simulación es de 127, al mismo tiempo existen límites en cuanto al número de bloques que pueden usarse de los diferentes tipos, según la siguiente tabla:

Tipo de bloque	*Número máximo utilizable*
F	8
P	25
W	10
T	5 con 25 intervalos de integración de retardo total
K	30
Q	10
H y Z	8
M	2
C, X, Y, B y V	Sin límite

FORMATO DE LAS INSTRUCCIONES

Las instrucciones correspondientes a los bloques que simulan las componentes físicas de los sistemas, constan en general de:

1. No. de referencia del bloque.
2. Código alfabético del bloque.
3. Especificación de la configuración del sistema.
4. Parámetros.
5. Condiciones iniciales.

Los números de referencia tienen que estar comprendidos entre 1 y 127 y no pueden repetirse.

La configuración del sistema, se especificará por medio de los números de referencia de los bloques que estén a la entrada de aquel a que corresponde la instrucción. Así la instrucción 5 C 1 2 significa que existe un sumador cuyo número de referencia es 5, cuyas entradas son las salidas de los bloques cuyos números de referencia son 1 y 2. En general pueden usarse indistintamente 1 ó 2 entradas. Cuando se use una sola entrada se especifica la segunda entrada por cero. El bloque *F* (Función Forzante) no tiene entradas por tanto no se requiere ninguna especificación de las mismas, mientras que en el bloque *X* (multiplicador)las dos entradas tienen que corresponderse con las salidas de otros tantos bloques.

Los parámetros del bloque aparecerán después de las entradas y su cantidad dependerá del bloque en cuestión.

Por último, aparecerán las condiciones iniciales del bloque a que corresponde la instrucción, en los casos en que se trate de un sistema dinámico de primer o segundo orden o de un retraso puro.

Cuando los datos finales de una instrucción sean nulos es posible su omisión, realizándose su anulación automática por medio del programa que compone el lenguaje. Así por ejemplo la instrucción 3 *P* 1 0 0 que corresponde a un sistema de primer orden que usa solo una entrada y que tanto su condición inicial como su parámetro son nulos, puede escribirse 3 *P* 1. Una excepción lo constituye el bloque *K* (Amplificador) en el cual si no se especifica la ganancia se hace igual a la unidad.

En las figuras 1b y 1c, aparecen las instrucciones correspondientes a la impresión de los resultados. La instrucción *L* se usa cuando se quieren obtener los resultados en forma de un listado de las variables dependientes contra el tiempo, pudiendo obtenerse hasta tres variables. La instrucción *G* se usa cuando se desea un gráfico de una variable dependiente contra el tiempo, obteniéndose también un listado con los valores numéricos correspondientes.

En cada programa tiene que aparecer una y sólo una instrucción de impresión, siendo necesario cerrarla con una letra cualquiera.

a) INSTRUCCION DE LOS BLOQUES

NUMERO DE REFERENCIA	CODIGO	CONFIGURACION	PARAMETROS	CONDICIONES INICIALES

b) INSTRUCCION DE LISTADO

NUMERO DE REFERENCIA	L	BLOQUES A LISTAR	INTER. DE IMPRESION	INTER DE INTEGRACION	TIEMPO FINAL

c) INSTRUCCION DE GRAFICO

NUMERO DE REFERENCIA	G	BLOQUES A GRAFICAR	VALOR MINIMO	VALOR MAXIMO	INTER. DE IMPRESION	INTER DE INTEGRACION	TIEMPO FINAL

Los números de referencia de las instrucciones de impresión no están sujetos a limitación alguna.

En ambos casos después del código del bloque, se especificarán los números de referencia de los bloques cuyos resultados se requieren, (hasta 3 cuando se usa *L*-listado y 1 cuando se usa *G*-gráfico).

En el caso de gráfico, es necesario especificar a continuación los valores mínimo y máximo que corresponderán a los límites del gráfico. La resolución del gráfico resultante será de 1/60 de la diferencia entre los valores máximo y mínimo.

Por último, en ambos casos se especificarán el intervalo de impresión de los resultados, el intervalo de integración a usar en la simulación, y el valor de la variable tiempo hasta el cual se desea correr el programa.

PROGRAMACION POR MEDIO DEL LENGUAJE

Para la mejor comprensión del método de programación se considerará un ejemplo.

Supóngase que se requiere simular el sistema de la Figura 2, en el cual:

1) La entrada al sistema R es un salto escalón de amplitud A.

   $R = A\,U\,(t)$, que se aplica con el sistema en reposo.

2) El control es proporcional más integral, es decir su ecuación diferencial es:

$$m = K_c\left(e + \frac{1}{T_i}\int e\,dt\right)$$

3) El proceso es un sistema de primer orden, cuya ecuación diferencial es:

$$X' + bX = m$$

Previamente a la elaboración del programa, se requiere la confección de un diagrama de bloques, usando los que componen el lenguaje.

Para la variable de entrada al sistema R se usará un bloque designado por el código F (Función Forzante). En este caso, como se trata de un salto escalón, todos los parámetros a excepción del A son nulos, teniéndose por tanto el bloque de la figura 3a.

En el punto donde se comparan los valores de R y X se utilizaría un bloque C (sumador) unido a un bloque (inversor), ya que una de las entradas es negativa. El diagrama de bloques sería el de la figura 3b.

Para el control se requieren los siguientes bloques:

dos bloques K (amplificador)
un bloque Y (nodo)
un bloque P (sistema de primer orden) y
un bloque C (sumador)

los cuales se enlazarán según muestra la figura 2c.

El primer amplificador simula la ganancia Kc del control. El nodo se requiere debido a que el bloque amplificador tiene una sola salida. El sistema de primer orden generará la componente integral del control; en este caso el parámetro a es nulo y como el sistema parte del reposo la condición inicial es nula también. Adicionalmente para la componente integral del control se requiere un amplificador de ganancia $1/T_i$. Por último se requiere un sumador para obtener la adición de los componentes proporcional (Kce) e integral ($Kc\frac{1}{T_i}\int edt$) del control.

Para simular el proceso se usaría un bloque P (sistema de primer orden) cuyo parámetro es b y su condición inicial nula (sistema que parte del reposo) figura 3d.

Con los diagramas anteriores conectados adecuadamente se obtiene el diagrama de bloques del sistema completo, figura 4. Se observará que en éste aparece, en adición a la designación de los bloques y los parámetros de éstos, un número de referencia que se utilizará en la identificación de los bloques al momento de preparar el programa. En este caso se han tomado los números de referencia consecutivamente a partir de 1, pero esto no es imprescindible.

A partir del diagrama de bloques se elaboran las instrucciones correspondientes a la simulación del sistema, según el formato de las mismas. Las instrucciones del ejemplo presentado son las siguientes:

Función forzante

1	*F*	*A*	0	0	0	0	0

Amplificadores

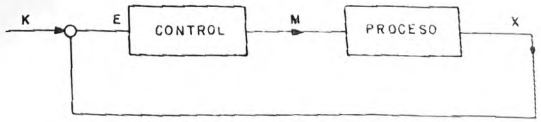
3	*K*	2	0	*Kc*
5	*K*	4	0	$1/T_i$

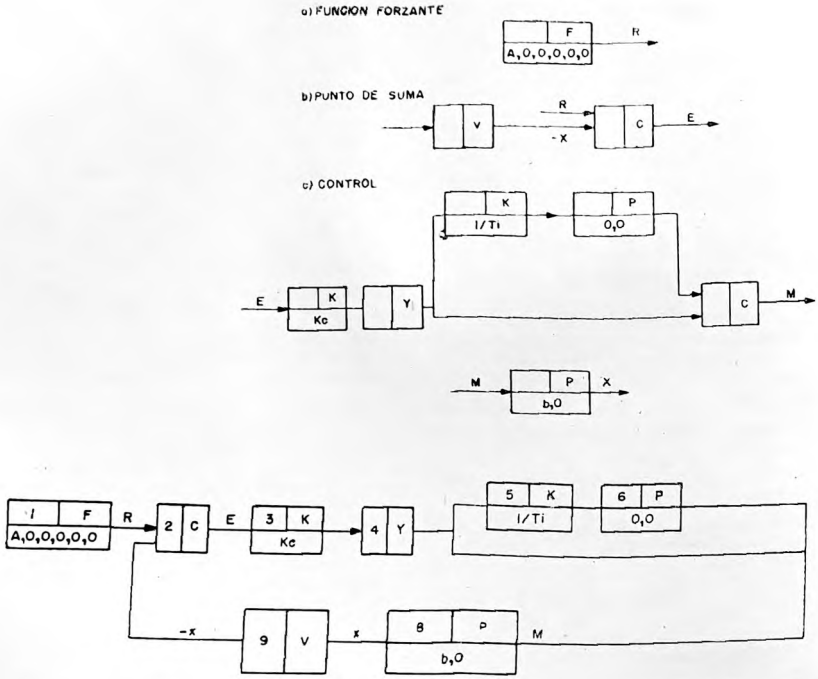
Sumadores, nodo e inversor

2	*C*	1	9	
7	*C*	4	6	
4	*Y*	3	0	
9	*V*	8	0	

Sistemas de primer orden

6	*P*	5	0	0	0
8	*P*	7	0	*b*	0





Adicionalmente se requiere elaborar la instrucción de impresión.

En el caso de que se requiera *listar* las variables *E*, *M* y *X*, la instrucción sería:

30 *L* 2 7 8 Intervalo de impresión, Intervalo de integración, Tiempo Final.

Si se deseara graficar la variable *X*, la instrucción sería:

35 *G* 8 Valor mínimo, Valor máximo, Intervalo de integración, Tiempo final.

El programa final siendo $A = 1.0$, $Kc = 0.25$, $T_i = 0.25$ y $b = 0.4$ es el siguiente.

Programa para la simulación de un Sistema de Primer Orden con control pi.

```
1    f    1
2    c    1    9
3    k    2    0    0.25
4    y    3
5    k    4    0    4
6    p    5
7    c    4    6
8    p    7    0    0.4
9    v    8
10   g    8    0    1.5    0.5    0.05    10.0    1
```

Los resultados de este programa aparecen en el Anexo No. 2.

ANEXO 2

RESULTADO DE LA SIMULACION DE UN SISTEMA DE PRIMER ORDEN CON CONTROL PI

t	8	0.00000	.02500	.50000
50002	.21273			
1.00005	.53104			
1.49992	.85903			
1.99980	1.12543			
2.49969	1.29158			
2.99957	1.35062			
3.49945	1.31993			
3.99933	1.23082			
4.49982	1.11821			
5.00030	1.01216			
5.50079	.93301			
6.00128	.88985			
6.50177	.88171			
7.00226	.90057			
7.50275	.93500			
8.00317	.97344			
8.50366	1.00678			
9.00415	1.02945			
9.50464	1.03967			
10.00513	1.03877			

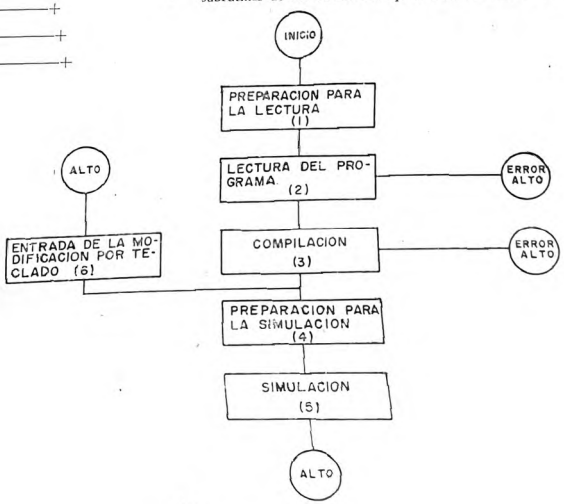
ESTRUCTURA DEL PROGRAMA QUE COMPONE EL LENGUAJE

La figura 5 muestra el diagrama de flujo del programa que compone el lenguaje.

La lectura del programa (bloque 2 de la figura) consiste en las operaciones de depositar en la memoria de la máquina, los datos contenidos en el programa. Para ello se utiliza además de una subrutina general de lectura, una particular por cada bloque que compone el lenguaje. La primera se encarga del direccionado de los datos correspondientes a la configuración del sistema a simular, mientras que las últimas direcciones, el depósito de los datos de parámetros y condiciones iniciales de los distintos bloques. Durante el proceso de lectura se detectan los errores de sintaxis que puede contener el programa pasando la máquina a la condición de ALTO DINAMICO, e imprimiéndose un número por el teletipo que corresponde al tipo de error detectado.

Una vez realizada la lectura de los datos se pasa al bloque de compilación (No. 3 de la figura), el que se encarga de ordenar adecuadamente la realización de las subrutinas de cálculo que se usarán en la simulación, según la configuración del sistema a simular. Durante la compilación se detectan los errores que puedan existir, en cuanto a bloques con más salidas que las permitidas, bloques sueltos, etc., pasándose en caso afirmativo a la condición de ALTO DINAMICO al igual que ocurre durante la lectura.

El bloque 5 del diagrama de flujo se compone también de una subrutina por cada uno de los bloques que compone el lenguaje, además de una subrutina general. Las subrutinas de los distintos bloques serán llamados en el



orden requerido atendiendo a la estructura del sistema que se simula. En este bloque están incluidas también las subrutinas requeridas para la impresión tanto de listado como de gráfico.

Para dar entrada por teclado a modificaciones, existen dos subrutinas, una para la modificación del intervalo de integración y otra para los parámetros de un bloque cualquiera. En ambos casos se pondrá la máquina en ALTO, y se oprimirá una recla antes de dar entrada a los cambios usando el teclado decimal.

El lenguaje emplea además las subrutinas de aritmética entera y flotante, junto con el intérprete y la entrada y salida elaborados por el Congreso de Investigación Digital, con algunas modificaciones ligeras que se le introdujeron.

En total se usan alrededor de 2,400 localizaciones de memoria para las instrucciones que componen el lenguaje, y el resto hasta completar los 4 k de la máquina, para depositar datos y resultados.

CONCLUSIONES

La redacción de este trabajo coincide con la terminación de la elaboración del Lenguaje, por lo que aún no se dispone de una experiencia de utilización del mismo, lo cual limita el alcance de estas conclusiones.

En las actuales condiciones se puede decir lo siguiente:

1. Aunque el tiempo de corrimiento de un programa es, en general, mayor con el Lenguaje que con una computadora analógica, el tiempo total no lo es, si se tiene en cuenta que no es necesario escalar en tiempo y en magnitud, no se requiere alambrar la máquina ni comprobar la operación de amplificadores, campos de integración, etc. Es preciso señalar que el Lenguaje se ha elaborado tendiendo a minimizar el número de localizaciones de memorias empleadas, tomándose el tiempo de solución como un elemento de segunda importancia. Dentro de la capacidad de memoria de la CID-201-A se pudiera aumentar la velocidad de ejecución, reduciendo la capacidad de cálculo del Lenguaje.
2. La repetibilidad de los resultados obtenidos por medio del Lenguaje es absoluta, lo cual no ocurre con la computadora analógica.
3. La capacidad de cómputo del Lenguaje corresponde a una computadora analógica de grandes dimensiones, siendo superior por ejemplo, a las tres máquinas analógicas de la Escuela de Ingeniería Eléctrica, la mayor capacidad de cálculo analógico del país.
4. La limitación actual en los periféricos de la CID-201-A limita las posibilidades del Lenguaje con relación a la analógica, ya que sólo se pueden obtener gráficos contra el tiempo y no hay posibilidad de observar los resultados en un tubo de rayos catódicos.
5. La elaboración del Lenguaje ha permitido entrenar a un grupo de alumnos de la Escuela de Ingeniería Eléctrica y a dos ingenieros de la misma, en las técnicas de programación en código de máquina, situándolos en disposición de abordar otros trabajos de programación.
6. El Lenguaje elaborado debe contribuir a ampliar la experiencia, en la utilización de la computadora digital en la solución de problemas concretos de la producción y el desarrollo.